SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

Course: Cloud Computing and Big Data Analytics (CSA1514) | CO2 (BL3) | Total Marks: 50

Name / Reg No: VIKRAM P(192511321) Signature: vikram

I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Question 1: Type-1 vs Type-2 Hypervisors

Problem interpretation: A datacenter hosting mission-critical applications needs high performance, strong isolation, high availability and centralised management. The hypervisor type must be chosen on those criteria.

Comparison

Aspect	Type-1 (Bare-metal)	Type-2 (Hosted)
Architecture	Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V, KVM, Xen).	Runs as an application on a host OS (e.g., VMware Workstation, VirtualBox).
Performance	Direct access to CPU, memory, NIC and storage; minimal overhead; supports hardware-assisted virtualization, SR-IOV, NUMA awareness.	Every I/O call passes through host OS and its drivers; extra scheduling layer causes higher latency and jitter; host OS tasks compete for resources.
Security	Small attack surface (thin, purpose-built kernel); no general-purpose OS services; strong VM isolation.	Large attack surface: host OS vulnerabilities, other host applications and malware can compromise all VMs.
Manageability	Centralised management (vCenter, SCVMM, OpenStack); live migration, HA, DRS, clustering, templates, automation APIs.	Managed per machine; no native clustering, live migration or HA; manual patching of host OS plus hypervisor.
Reliability	Fewer layers means fewer failure points; designed for 24x7 operation.	Host OS crash or reboot or update takes all VMs down.
Typical use	Production datacenters, cloud providers.	Development, testing, training, desktops.

Analysis and Justification

Mission-critical workloads cannot tolerate the extra latency, shared-fate failures and larger attack surface of a hosted design. Type-1 removes the intermediate OS layer, which gives near-native performance and predictable resource scheduling. Its minimal code base lowers the number of exploitable vulnerabilities, and features such as live migration, high availability, fault tolerance and automated resource scheduling are essential for meeting uptime SLAs.

Conclusion: Type-1 hypervisor is the better choice for a cloud datacenter hosting mission-critical applications. Type-2 is acceptable only for development/test and demonstration (as in this lab using VMware Workstation Pro).

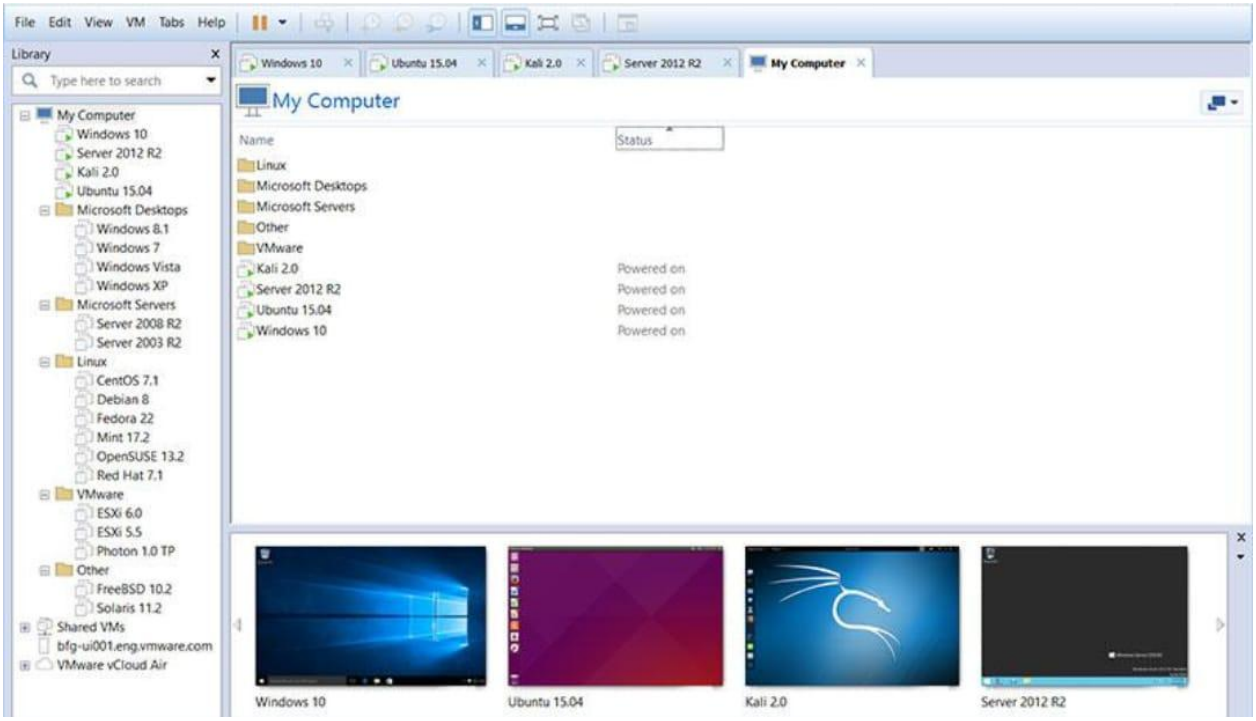


Figure 1: Library pane in VMware Workstation Pro showing multiple VMs (analogous to multiple instances/regions).

Question 2: Workload Isolation (Finance, Healthcare, Student Portals)

Problem interpretation: Three tenants with different sensitivity levels (financial data, protected health data, student records) share the same physical host. The provider must prevent data leakage and interference between them.

How virtualization architecture supports isolation

- **Hardware-level isolation:** The hypervisor gives each VM its own virtual CPU, memory, virtual disk and vNIC. Memory management (shadow page tables / EPT / NPT) prevents one VM from reading another VM's memory.
- **Fault and security isolation:** Each workload runs in a separate VM with its own guest OS, so a compromise or crash in one guest does not directly affect others.
- **Network isolation:** Separate VLANs / virtual switches / port groups and virtual firewalls (micro-segmentation) keep finance, healthcare and student traffic apart.
- **Storage isolation:** Separate datastores/LUNs or encrypted virtual disks per tenant, with access control, protect data at rest.
- **Resource isolation:** CPU/memory reservations, limits and shares stop a noisy VM from starving others (resource isolation).
- **Policy-based placement:** Dedicated hosts or clusters, affinity/anti-affinity rules and VM encryption can be applied for regulated workloads such as healthcare (HIPAA) and finance (PCI-DSS).

Two major attack surfaces

1. The hypervisor itself (VM escape / hypervisor compromise). A vulnerability in the hypervisor, emulated devices or virtual drivers can let an attacker break out of one VM and control the host, thereby gaining access to all co-located tenants. Mitigation: timely patching,

minimal hypervisor footprint, disabling unused virtual devices, secure boot, and hardened host configuration.

2. Shared resources and virtual network / management plane. Side-channel attacks (cache timing, Spectre/Meltdown class), VM-to-VM traffic across virtual switches, VM sprawl, snapshot/image theft and compromised management consoles or APIs can expose data. Mitigation: micro-segmentation, VLAN separation, dedicated hosts for sensitive tenants, encrypted snapshots, RBAC with multi-factor authentication and audit logging for the management interface.

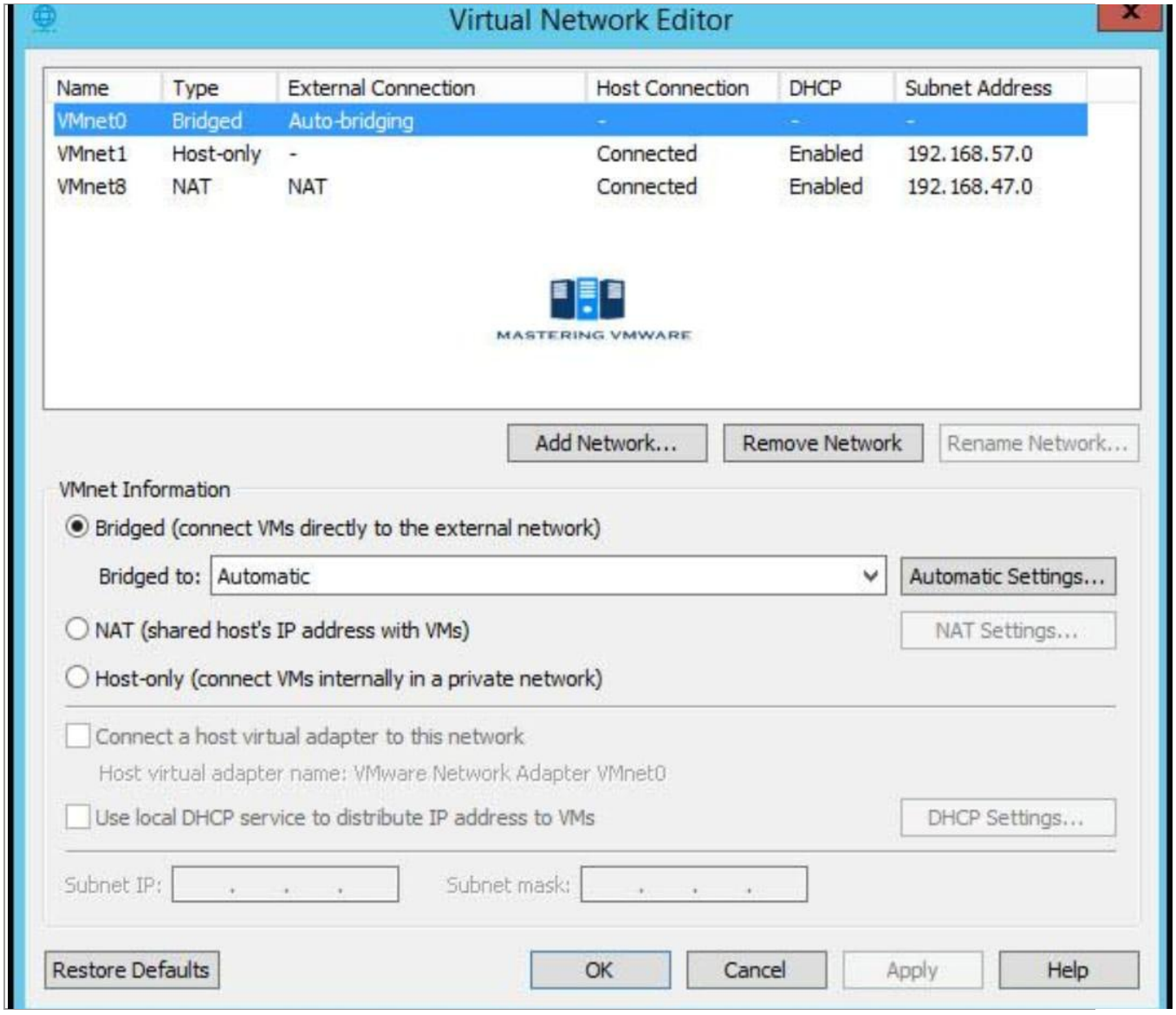


Figure 2: Virtual Network Editor showing separate Host-only / NAT / Bridged networks for isolating workloads.

Question 3: VM Host Utilization Analysis

Snapshot: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s.

Metric	Interpretation	Likely Bottleneck / Impact
CPU 92%	Well above the usual 70-80% safe ceiling; little headroom for bursts.	CPU overcommitment; high CPU ready time causes VMs to wait for scheduling; slow application response.
Memory 88%	Close to exhaustion.	Risk of ballooning, host swapping and page sharing/compression, which push extra load onto storage and CPU.
Storage I/O latency 35 ms	Healthy datastores are typically below 10-20 ms.	Storage contention (too many VMs on one datastore/LUN, slow disks, insufficient IOPS, possible swap traffic).
Boot time 170 s	Normal boot is roughly 30-60 s.	Boot storms and disk read latency plus CPU/memory contention during VM start-up.

Explanation of likely bottlenecks

The primary bottleneck is storage I/O combined with resource overcommitment. High memory usage triggers swapping/ballooning, which creates extra disk I/O; already slow storage (35 ms) then delays boot reads and application I/O; and a CPU at 92% prolongs guest start-up processing. The factors reinforce each other, so boot time rises sharply (cause-effect chain: memory pressure, then swap I/O, then latency increase, then slow boot and CPU wait).

Recommended corrective actions

- **Load balancing:** Use live migration / DRS to move VMs to less loaded hosts and rebalance the cluster.
- **Right-sizing:** Right-size VMs (reduce over-allocated vCPUs and RAM), remove idle VMs, and set reservations only where needed.
- **Capacity expansion:** Add hosts or RAM/CPU capacity; enable auto-scaling for peak workloads.
- **Storage improvement:** Move I/O-heavy VMs to SSD/NVMe tiers, spread VMs across multiple datastores, enable Storage DRS / storage I/O control, and check multipathing and RAID configuration.
- **Faster boot:** Use linked/thin clones or cached templates, stagger VM power-on, and use SSD-backed boot datastores.
- **Monitoring:** Set alerts (CPU > 80%, memory > 85%, latency > 20 ms) and review trends for capacity planning.

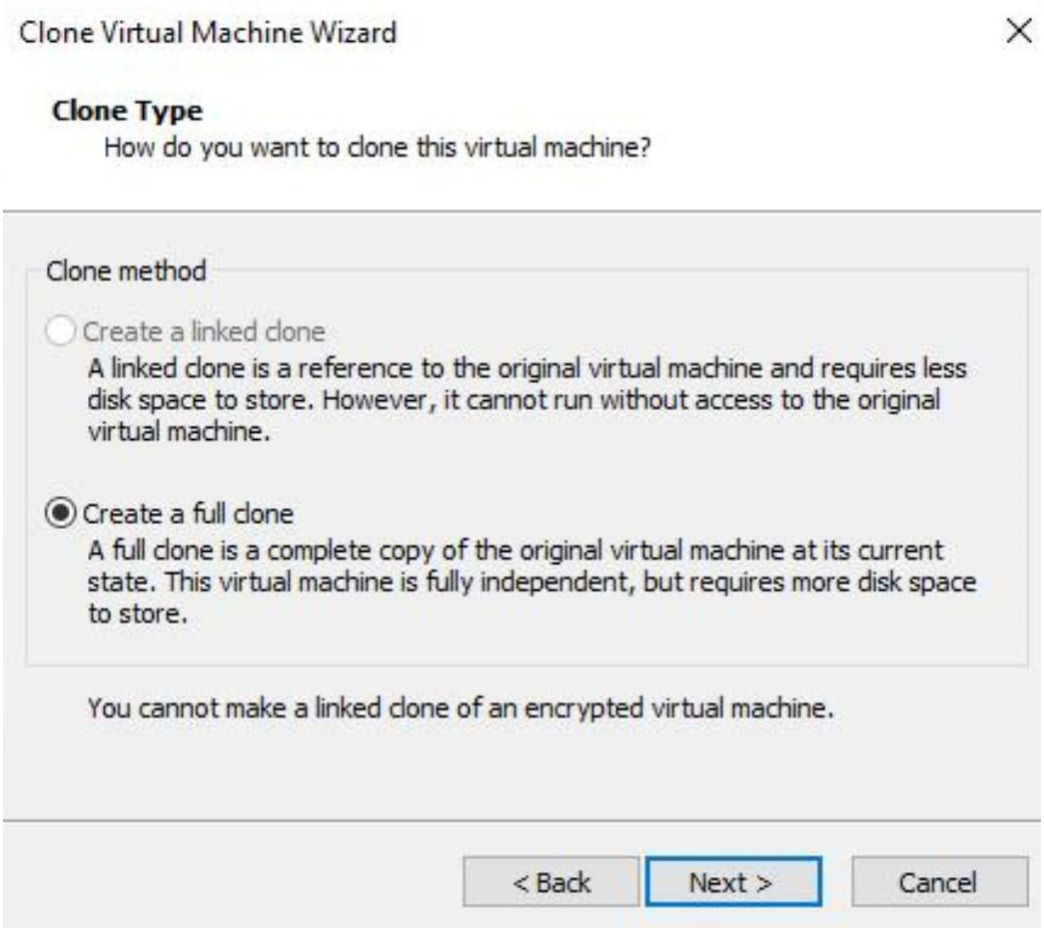


Figure 3: VM → Manage → Clone, demonstrating replicas/scale-out to relieve an overloaded host.

Question 4: Software Defined Datacenter (SDDC) vs Traditional Datacenter

A Software Defined Datacenter virtualizes compute, storage and networking and manages them through software with policy-based automation and APIs. A traditional datacenter ties each function to dedicated hardware managed manually, which makes changes slow.

Dimension	Traditional Datacenter	Software Defined Datacenter
Compute	Physical servers per application; provisioning takes days or weeks; low utilization.	Server virtualization (VMs/containers); provisioned in minutes from templates; pooled resources, live migration, auto-scaling; high utilization.
Storage	Fixed SAN/NAS arrays; manual LUN provisioning; vendor lock-in.	Software-defined storage (e.g., vSAN, Ceph) pools commodity disks; policy-based provisioning, tiering, replication, snapshots.
Network	Hardware switches/routers configured box by box; VLAN changes by hand.	Network virtualization/SDN (e.g., NSX, OpenFlow): separate control and data plane, logical switches/routers, micro-segmentation, programmable via API.
Management	Siloed teams and tools; manual and error-prone.	Unified orchestration and automation; infrastructure as code; self-service portal.

Dimension	Traditional Datacenter	Software Defined Datacenter
Cost model	High CapEx; over-provisioning.	Commodity hardware; pay-as-you-grow; lower OpEx.

Analysis of agility improvement

- **Speed of provisioning:** New environments (VMs, volumes and networks) can be created, cloned or destroyed in minutes through templates and APIs, supporting faster releases and DevOps.
- **Elasticity and mobility:** Resources scale up or down on demand (elasticity) and workloads move between hosts without downtime.
- **Consistency and automation:** A single policy engine applies the same security and QoS rules across compute, storage and network, reducing configuration drift and human error.
- **Resilience and portability:** Disaster recovery, backup and hybrid cloud connectivity are easier because the infrastructure is abstracted from hardware.

Conclusion: By abstracting hardware into software-controlled pools, SDDC removes manual bottlenecks and enables rapid, repeatable and policy-driven change, giving much higher agility than a traditional datacenter.

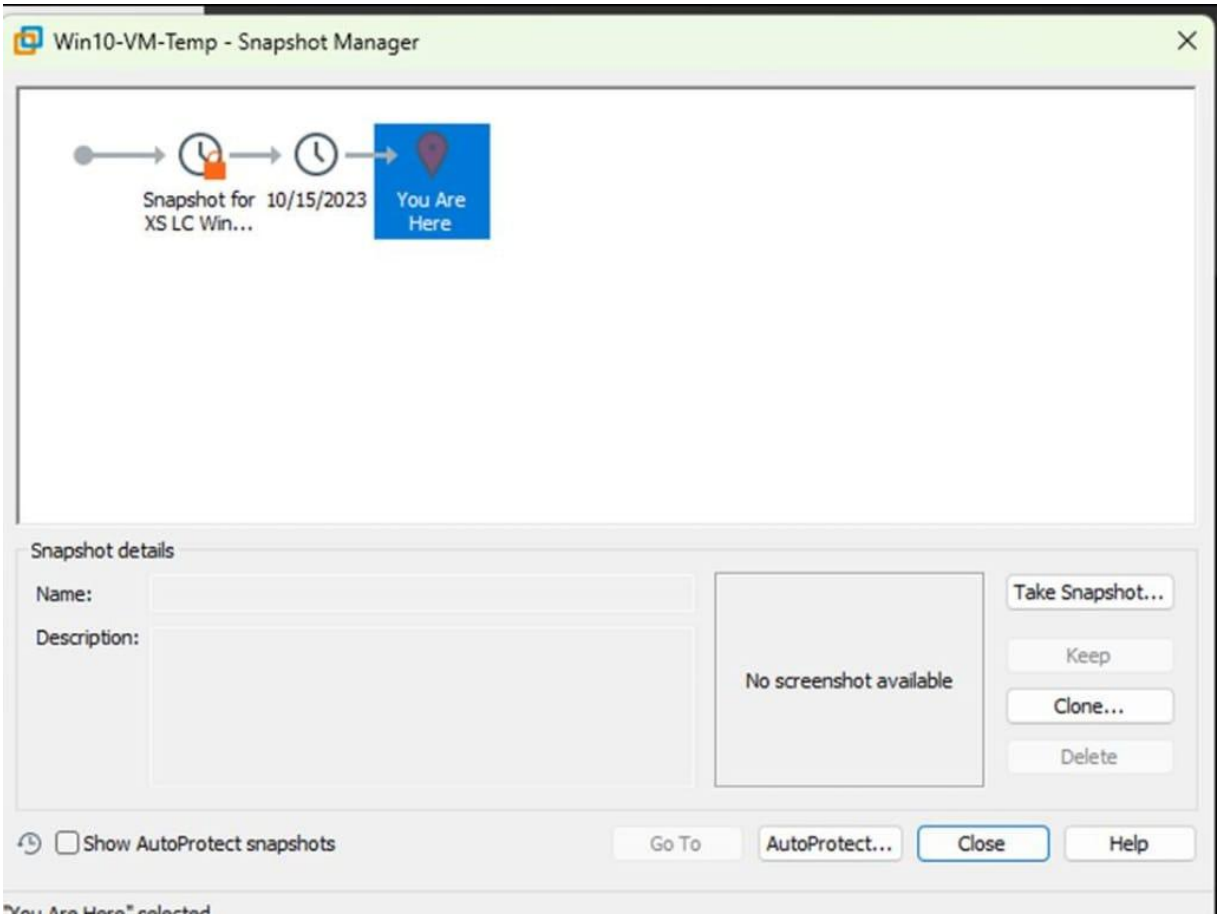


Figure 4: Virtual Network Editor (NAT / Host-only / Bridged) showing software-defined networking.

Question 5: Multi-Cloud Federation Identity Mismatch

Analysis of the issue

Each cloud provider (e.g., AWS, Azure, GCP, private cloud) maintains its own identity store, user naming format, attribute schema, roles and token format. When clouds are federated without a common trust framework, the same user can appear under different identifiers (e.g., user@a.com vs UID123), roles do not map, and tokens are not mutually understood. This causes authentication failures, authorization errors, duplicate or orphaned accounts, and also encourages insecure workarounds such as shared credentials.

Root causes: (1) no common identity standard, (2) no agreed attribute/role mapping, (3) lack of cross-provider trust between identity providers, (4) inconsistent lifecycle management (deprovisioning), and (5) different privacy and compliance requirements.

Proposed secure identity and privacy design

- **Federated identity with trust relationships:** Use a central or trusted Identity Provider (IdP) such as Azure AD/Okta/Keycloak; each cloud acts as a Service Provider (SP) and trusts it.
- **Open standards:** Use SAML 2.0 for enterprise web SSO and OpenID Connect / OAuth 2.0 for API and modern applications, with signed and short-lived tokens (JWT).
- **Identity normalization:** Use SCIM for automatic provisioning and deprovisioning of accounts across providers; adopt a unique, persistent identifier (e.g., immutable GUID).
- **Attribute and role mapping:** Create a common attribute schema and map IdP groups/claims to each cloud's IAM roles using RBAC/ABAC with least privilege.
- **Security controls:** Enforce MFA, conditional access, certificate/key rotation, TLS everywhere and centralized audit logs/SIEM.
- **Privacy:** Release only the minimum required attributes (data minimization), use pairwise/pseudonymous identifiers, user consent, and encryption of personal data; comply with GDPR/DPDP-type regulations and data-residency rules.
- **Governance:** Use periodic access reviews, federation metadata exchange and automated revocation when a user leaves.

Flow

User logs in to the trusted IdP, then the IdP authenticates (MFA) and issues a signed token/assertion with minimal claims, then Cloud A/B/C validate the signature, map claims to local roles and grant least-privilege access, then all access is logged and audited.